



staged AI-agent delivery pipeline · Claude Code & Codex

# tms-pipeline

Пошаговый процесс для AI-агентов: от задачи до проверенного кода

**tms-pipeline** — это дисциплина для AI-агентов: она проводит одну задачу от тикета до проверенного кода, готового влиться в основную ветку, и держит рабочую память агента чистой на каждом шаге. Решения остаются за вами — вы контролируете каждый шаг. Работает через **скиллы** — команды вида `/tms-research`, которые вы даёте агенту, чтобы он выполнил один шаг. Открытый код, документация и установщик: [github.com/TmsNine/tms-pipeline](https://github.com/TmsNine/tms-pipeline).

## Какую задачу решает

Когда агенту дают большую задачу одним промптом, он переполняет рабочую память, теряет ранние инструкции, пропускает редкие случаи и выдаёт код, который трудно проверить. **tms-pipeline** разбивает задачу на восемь коротких стадий: на каждой остаётся отдельный документ и точка проверки. Поэтому любой шаг можно отследить, перепроверить и безопасно влить в живой проект.

## Как это работает

- **Восемь стадий, на выходе каждой — отдельный документ.** Нумерация идёт от 00 до 06, но между 02 и 03 есть вставная стадия 02b — поэтому стадий восемь, а не семь: 00 Ticket → 01 Research → 02 Design → 02b Gap-audit → 03 Plan → 04 Implement → 05 Test → 06 Review-gate. Семь стадий — это обдумывание на бумаге ( `.md` ); код пишется только на одной, 04.
- **Реализация силами mob** — команды из нескольких агентов-ролей. Главный агент раздаёт работу и сам код не пишет; на каждый небольшой кусок (волну) он запускает разработчика и проверяющих — до пяти ролей: разработчик, тестировщик, архитектор, безопасник, ревьюер. Дальше волна не идёт, пока все проверки не зелёные.
- **Аудит пробелов.** Ещё до кода другой агент намеренно ищет в дизайне дыры и оценивает каждую по тяжести: A (блокер), B (чинимая авария), C (мелкая доработка), D (теоретическая). Смотрит под разными углами — безопасность, параллельная работа, UX, эксплуатация, целостность данных, приватность.
- **Профили эскорта A / B / C** — способ не звать лишних проверяющих: по умолчанию работает дешёвая модель, а дорогие проверки подключаются только когда задача того требует. На простых волнах это экономит около 40 % стоимости.
- **Ничего не теряется.** Всё, что нашли по ходу, обязательно попадает либо в бэклог (список будущих задач), либо в план запуска.

## Установка и онбординг

Установка разделена на две части. Сначала простой установщик ( `npx tms-pipeline` или плагин Claude Code) ставит скиллы и кладёт стартовый `AGENTS.md` — файл с настройками проекта; о проекте он не расспрашивает. Затем скилл `/tms-init` внутри Claude Code или Codex сам читает ваш репозиторий — команды тестов и сборки, формат тикетов, документацию, бэклог — и заполняет настройки, спрашивая только то, что не смог вычитать. Документация живёт в вашей базе документации (Obsidian, Notion и т.п.), а сами задачи выполняются в репозитории. Два языка: русский — основной, с ним сверяют английскую версию.

## Для кого

Командам и соло-разработчикам, которые ведут реальный проект с помощью AI-агентов и хотят стабильного, повторяемого качества вместо разовых удач — так, чтобы каждый шаг оставался под контролем человека.